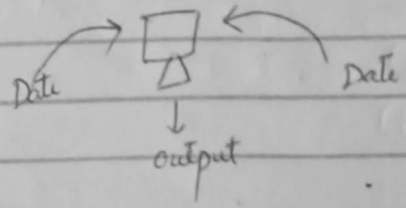
Machine Learning

Def:-
ML is all about teaching comp to learn from Data.
ex: just like human learn from experience.

→ How Netflix work.
→ How google translate.

- بچہ کیسے 1 سال میں بولنا سیکھتا ہے 1 ← long



"Instead of programming every step, we give machine lots of data & let them figure out pattern on their own"

- اس میں ہم machine کو data دیتے ہیں اور وہ pattern خود سیکھتی ہے

Use — Real life ex of ML

- Youtube Recommendations:
  learn from your watch history
- Spam detector in Gmail:
  learn pattern in spam email
- Voice assistance:
  learn how you speek
- Self Driving Cars:
  learn to identify stop signs, pedestrains & roads

• Face Unlock on Phones → Scan your face.

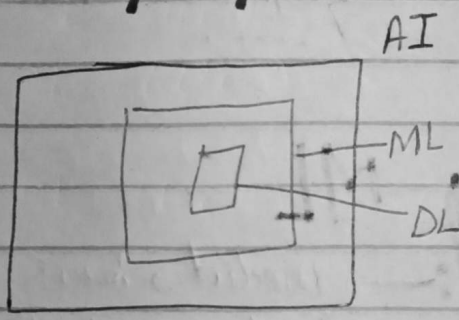
Traditional Programing :-

- Give rules + Data → get result
- Manual logic writing.

Machine Learning :-

- Give Data + result → get rules (model)
- Learn pattern automatically.

AI / ML / DL



AI: Any system that mimics human intellegence

ex:-

- Playing chess like a human
- Talking to Alexa
- Driving car on its own

ML:- ML is subset of AI
this is "Where" Machine learn from Data.

ex: • Youtube reeem
• email Spam

DL:- DL is a subset of ML - it uses something called neural network, which is inspired by the human brain.

- face recognition
-
-

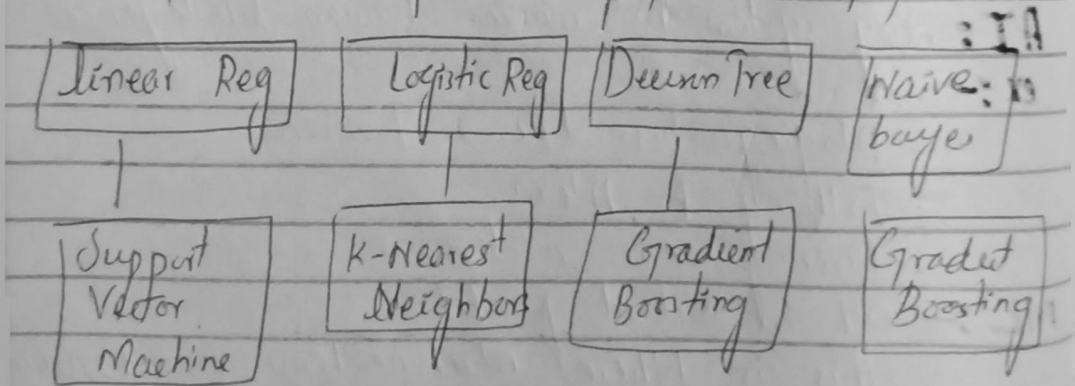
AI / ML / DL

AI is the big picture, ML is the brain that learn from data & DL is the super-power brain build to handle really tough problems!

- Note : Data Science is a field. Inside you learn ML / DL / ---.

Type of ML

① Supervise ML:— • Predict, house
• Input & output



③ Unsupervise ML:— find pattern, Group Similar Item.
• Only Input Data

group کر pic جیسی search کرتا ہے۔—: Note

③ Reinforcement Learning :-

↳ Learning by trial & error.

ex :- Online games.

EDA

Let see the steps involved in Making ML model.

① Problem definition
② Data collection
③ EDA → Exploratory Data Analysis
④ Data preprocessing / cleaning
⑤ Feature selection & Engineering
⑥ Split the Dataset
⑦ Model selection.
⑧ Model Training
⑨ Model Evaluation
⑩ Hyperparameter Tuning
⑪ Model Testing / Validation

EDA

EDA stands for Exploratory Data Analysis - it's the step where you explore the data to :

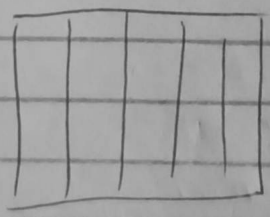
- Understand it

- Discover pattern
- Spot anomalies → Null value or ? model کو خراب
- Generate insights
- And decide what to do next.

"Think of EDA as detective work — you are looking at your data with curiosity before building any models or doing fancy transformations.

EDA steps :—

1. Viewing the Data :
   - head(), tail(), shape, info()
   - → What columns do I have?
   - → What types of data?
2. Summary statistics
   - mean, medium, mode, std, min, max, quartiles.
   - → Helps understand spread & Central tendency.



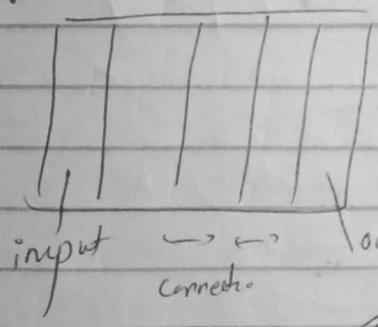
کسی بھی mod / med / mean کالم کے لیے

3. Value Counts :—
   - How many unique values in a Coulmn?
   - Great for categorical columns.

(4) Missing Value Analysis?
Where are the gaps? What percent of the data is missing?

(5) Visualizations:
- Histograms → Distribution of Values.
- Boxplots → Outliers & spread.
- Bar plots → Comparisons of categories.
- Correlation heatmaps → linear Relations b/w numeric features
- Scatter plots → bivariate relationships.

(6) Target Variable Exploration:
- How does your output (like 'charges' in your Dataset) relate to other variables?



output ← step ← ... ← input

EDA Importance:
- You can't clean or preprocess what you don't understand.
- It helps you identify mistakes, biases or limitations in the data.
- It guides the direction of your data cleaning & feature engineering.
- It gives your audience or stakeholders a story or overview of what the

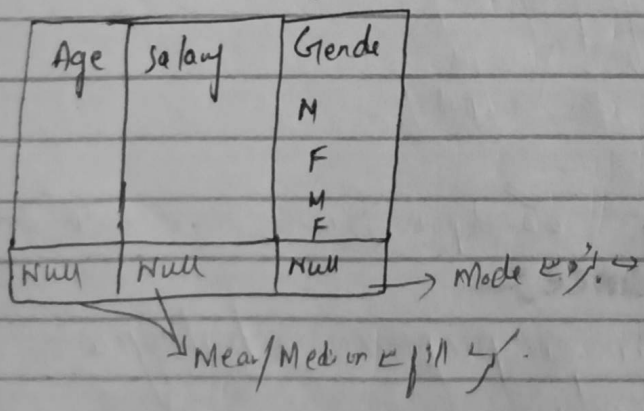
the data is telling you

② Data Cleaning

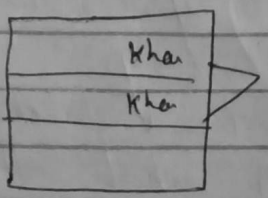
① Handling Missing Values :-
Check which colums have Missing Values. (nulls).

② Strategies to Handle :-
- Drop Missing rows / columns (only if very few).
- Impute with :-
  - Mean / Median → for Numerical Data
  - Mode → for categorical Data
- Advance : Linear Reg, KNN or Interpolation (for future Learning)

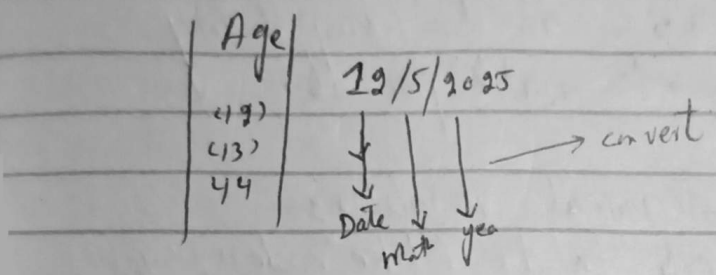
(ex)



③ Remove Duplicates :-
→ Detect & drop exact duplicate rows



③ Fix Data Types:-
Convert wrong types
e.g.s
numbers stores as string, data as text)



④ Handle Inconsistent Categories.
Clean up Categorical Values like
* "Male", "male", "MALE"
↓
should all become "male"
* "Yes", "yes", "Y"
↓
unify to 1 format

⑤ Detect & Handle Outliers:-
- Use boxplot, IQR, Z-score
- Handle by:
  * Removing (if clearly wrong)
  * Capping (e.g., to 95th percentile)

⑥ ~~Fix~~ Fix Logic or Domain Errors
→ E.g., age = -5 is Invalid or BMI=150 likely an error
→ Can replace with mean, median or remove

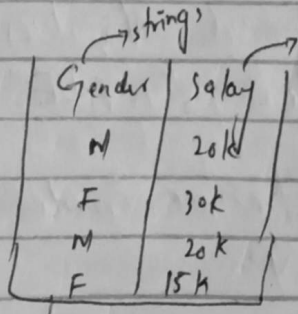
Note: 80% work is Data Cleaning.

③ Data Preprocessing

To prepare clean data so it can be analyzed or used in a ML model."

⇒ In Data cleaning is about fixing mistakes, Data processing is about transforming Valid data into usable format.

① Encoding Categorical Variables :-
Convert text labels (like "male", "yes") into numbers.



→ machine को read करने के लिए strings को Numeric में convert करना होता है

M → 0
F → 1

2 Common Method

① Label Encoding
Ordinal
(1)
Good for ordered Categories like "Low", "Medium", "High"
• Converts each category into Seprat binary columns (0

② One-Hot-Encoding
Nominal.
(2)

- Convert each category into seprate binary coulmn (0 or 1)
  For non-ordered categories like region

ex :-

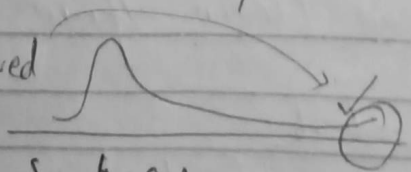
Reg	Gend
N	M
S	F
E	M
E	F
N	
S	
W	

N ⟶ 0
S ⟶ 1
W ⟶ 2
E ⟶ 3

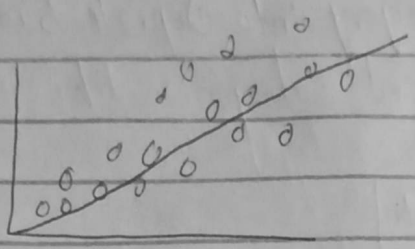
ML new - ہر ایک کو 1, 0 یا 3, 2 کی

Region North	Region south
0	0
1	1
0	0
1	1

(2) feature Transformation :- (log, Square root)
used to handle skewed data, like right-skewed or left skewed data



(3) feature Scaling :
(Normalization or Standardization)
Bring numerical values to the same scale - especially useful for distance-based algorithms.



Age	salary
23	36000
26	8000
34	16000

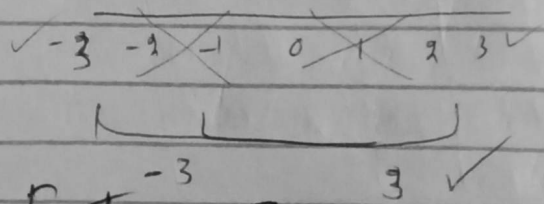
→

Age	salary
0-1	0-36
0-23	0-80
0-34	0-16

Data Preprocessing

Normalization (Min / Max Scaling):
Scale value b/w 0 & 1

Standardization (z-score Scaling):
Transforms data to have mean 0 & std 1

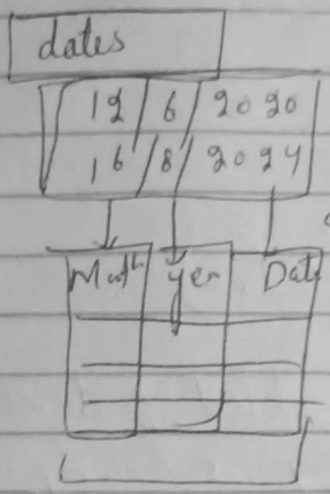


(4) Feature Engineering :-

Creating new features or transforming existing ones to expose useful pattern that ML models can learn from

Why do we need it?

Because ML models don't know domain logic — we have to give them the right signals



⇒ accuray 86%.

⇒ accuray 90%.

Note: Accuray Must be 80%.

Common Feature Eng Techniques in ML:—

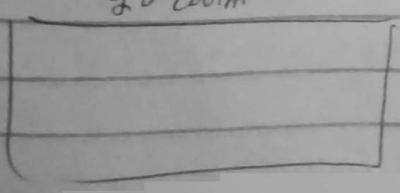
1. Mathematical Combination
2. Target-Based flags.
3. Binning (when it helps)
4. Time based features (if time exists)

(1) feature Selection

Selecting the most useful features & removing the rest.

Why is it important?

→ Reduce noise & overfitting
→ Speed up training
→ Improve model accuracy
→ Make mode Interpretation easier (with)



accuray 62%.

- 4 use of colm 12 to 28 f.

* feature Selection

(1) Filter Methods (pure statistics)
- Correlation Matrix
  Remove highly correlated features.
- Chi-square test (categ vs categ)
- ANOVA / F-test (numerical vs cat target)

(2) Embeded Methods (Selection build into the Model)

- Lasso Regressor → shrinks coefficient to 0
- Tree-based Models (Random forest, XG Boost)
  → feature Importance scores

(3) Handle Inconsist Categories :
- Clean up Categorical value like:
  "Male, male, MALE → should all become male

EDA

- first five row لیکن ؟
  df.head.
- Data کا shape لیکن ؟
  df.shape
- کتنا Null Value ہے یا Dtype ہے ؟
  df.info
- Mean / Median / Mode ؟ / max / 25% . 75% / 50% ؟
  df.describe()
- کتنی null value ہے ؟
  df.isnull().sum()

Visualization

- df.columns → output :- Index ['age', 'sex', 'bmi', chid, smoker, region, charges
- ایک ایک کالم لے کر اس کا نام لیں

```
numeric-columns = ['age', 'bmi', children, charges']
for col in numeric-columns:
    plt.figure(fitsize = (6,4))
    sns.histoplot(df[col]), kde=True, bin=2
```

curve

Data Analysis

- ڈیٹا کو زیادہ تر part میں 20 ڈیٹا

* sns.countplot(df['children'])

form ← chart ← bar ← count ← Data ← category ←

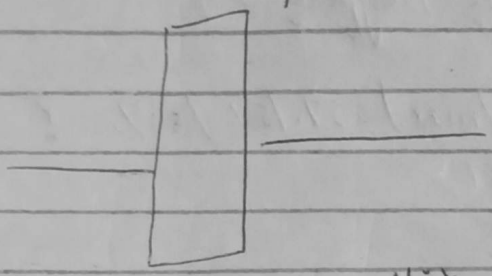
Gender
M
F
M
M

Male → 3 bar لیں گے
Female → 1 لیں گے

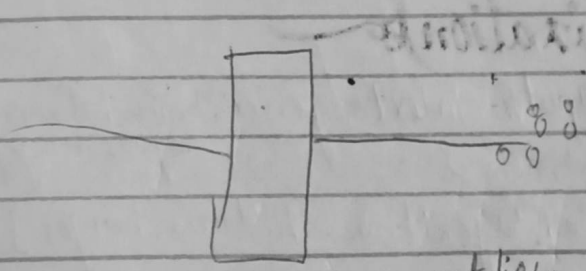
age sex smoking

* Input and output ko connect karke analysis karte hain.

```
for col in numeric_columns:
plt.figure(figsize=(6,4))
sns.boxplot(x=df[col])
```



No outlier ✓



outlier ✓

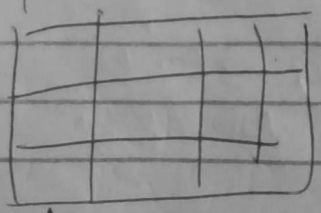
* Co-relation ?

```
plt.figure(figsize=(8,6))
sns.heatmap(df.corr(
```

add

numeric=True | annot=True

output



*1) Data cleaning & Processing

(1) df_cleaned = df.copy()
(2) df_cleaned.head() # first five rows
(3) df_cleaned.drop_duplicates(inplace=True)
(4) df_cleaned.isnull().sum()
(5) df_cleaned.dtypes
(6) df_cleaned['sex'].value_counts()
output => Male = 23
Femal = 32
mal = 3
(7) df_cleaned['sex'] = df_cleaned['sex'].map({"male":0, "female":1})
df_cleaned.head()

---

df_cleaned['smoker'].value_counts()
Output:
smoker <— Yes =
No =

convert کرنے کے لیے 1, 0 میں easily
just میں NOO کے لیے NO

df_cleaned['smoker'] = df_cleaned['smoker'].map({'male': 0, 'female': 1})
no yes

change کرنے کے لیے column کا نام

df_cleaned.rename(columns = {
'sex': 'is_female',
'smoker': 'is_smoker'}, inplace=True)

- Dummies میں

```
df_cleaned = pd.get_dummies(df_cleaned, columns=
                ['region'], drop_first=
df_cleaned.head()
```

output

Region	→	Nort	South	East	west

show 3 number ←

```
df_cleaned = df_cleaned.astype(int)
df_cleaned.
```

Numeric 4, Numeric 3 Machine 3 Note

Feature Engr...ing & Extraction

~~df_cleaned~~

```
sns.histplot(df['bmi'])
```

```
df_cleaned['bmi_category'] = pd.cut(
        df_cleaned['bmi'],
        bins = [0, 18.5, 24.9, 29.9, float('in
        labels = ['Underweight', 'Normal', 'Overw
)
```

Research

Feature Scaling

```
From sklearn.preprocessing import StandardSc
Cols = ['age', 'bmi', 'children']
scaler = StandardScaler()
```

standard scales

$z = \frac{x - \text{mean}}{\text{std}}$

orignal — colm ka average — st dev

mean = $\frac{19 + 21 + 24}{3} = \frac{64}{3} = 21.67$

std dev = $\sqrt{\frac{(19-21.67)^2 + (18-21.67) + \dots}{3}}$

```
df_cleaned[cols] = scaler.fit_transform
                   (df_cleaned[cols])
```

اس سے سارے value 0 اور 1 کے درمیان آ گئی۔

Feature Selection / Extraction :-

```
from scipy.stats import pearsonr.
Selected features = [ _ _ _ _ _
 _ _ _ _ _ _ _ _ _
 _ _ _ ]
```

یہ relation کرے گا Co-relation سے charges کے ساتھ ان سب کے
Dictionary کی طرح create کریں گے

```
Correlations = {
    feature : person(df_cleaned[feature],
                    df_cleaned['charges'])[0]
    for feature in selected_features.
}
```

lower سے highest کرنے کے لیے sort ←

```
correlation = df.sort_values(by='—',
                             ascending=False)
```

—— (Co-relation) ——

+1 ⟷ −1

~~0 0.7~~

- 50% ⟷ −50% ✓
- 20 ⟷ 0.50
- 10 ⟷ 0.20 → use کرنا ہے

(Chi 2 test)

Compare ... category ...

```
from scipy.stats import chi2_contingency
import pandas as pd.

alpha = 0.05
```

... step next ... Creat ... bins 4 ←

p < alpha ⇒ reject

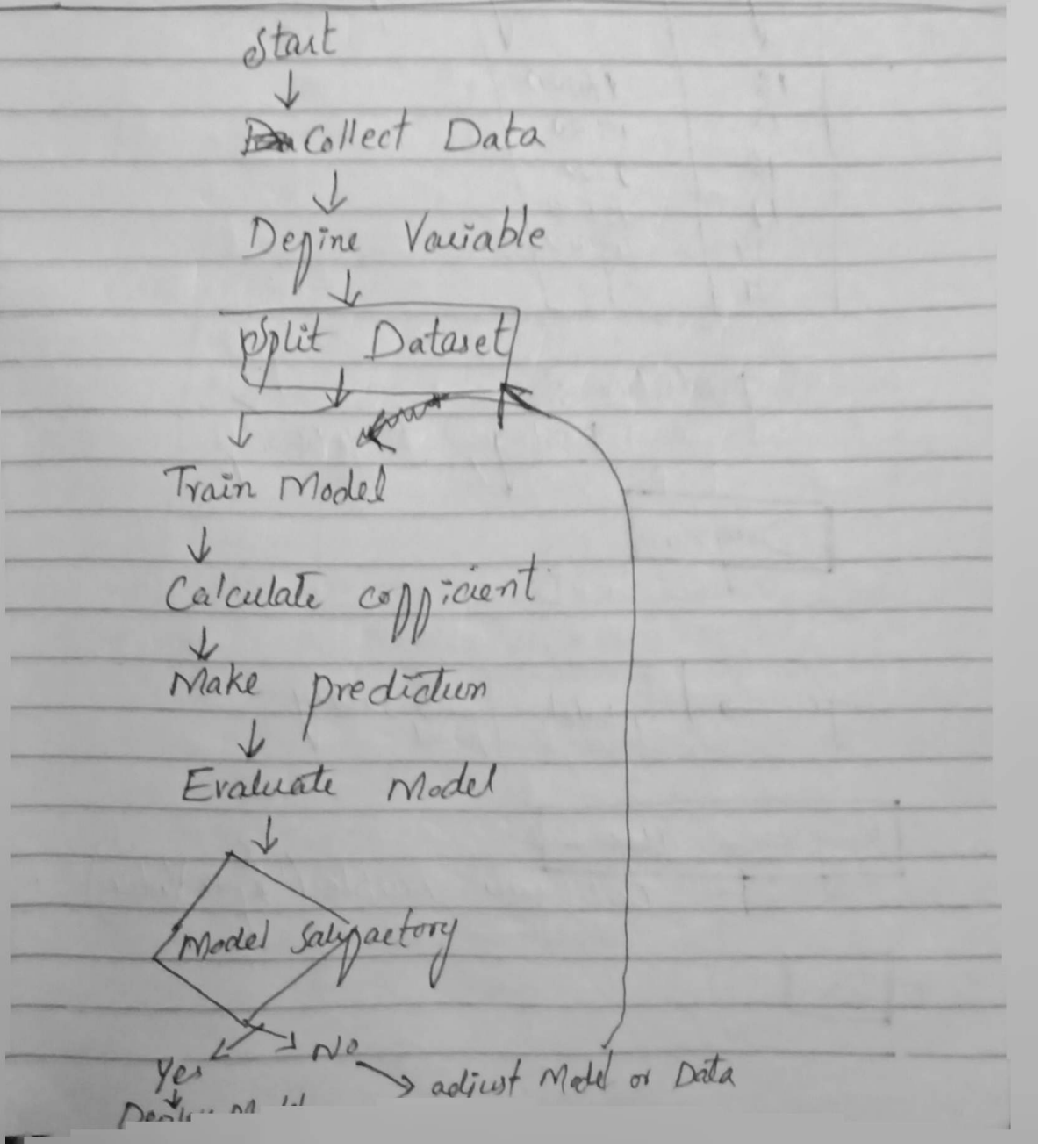
Lec : 02

Supervise Learning

↓

Regression → predict continous Var

1. What is Reg Alg
2. Reg Alg
3. Model Evaluation Tech
4. Overfitting & Underfitting
5. Hand on projects.



What is Regression ?

Regression is a way to predict a number
It helps us find out how one thing changes when something else changes.

2 variable → Age, Salary

Input → Age | Salary → output

Age	Salary
12	16000
13	17000
16	2000
17	42000
34	45000
36	?

predictor ?

↑ Salary, ↑ Age ⇒ ↑

Diagram

Age → Model → predict → Salary

Regression Use

for Continuous Vaible (Not fix Value)

Ex

* Input : Temp (گرمی کتنی ہے)
* Output : Lemonade جتنی گرمی ہوگی اتنے گلاس
ice کتنی ڈالنی ہے۔

Result ← جس میں model + input کو correct output (Label) دیتے ہیں
model سے کیا Relation بنتا ہے

EX

Temp (C°)	ice cubes
20	2
25	4
30	6
35	8
⋮	⋮

Concept :-

Term	Meaning
Sup ML	Input + Output دونوں دیا جاتا ہے
Feature	Temprature
Label	Ice Cubes
Type	Regresion (continous output)

Regression :-
"Regression is about finding pattern in numbers so we can make smart predictions."

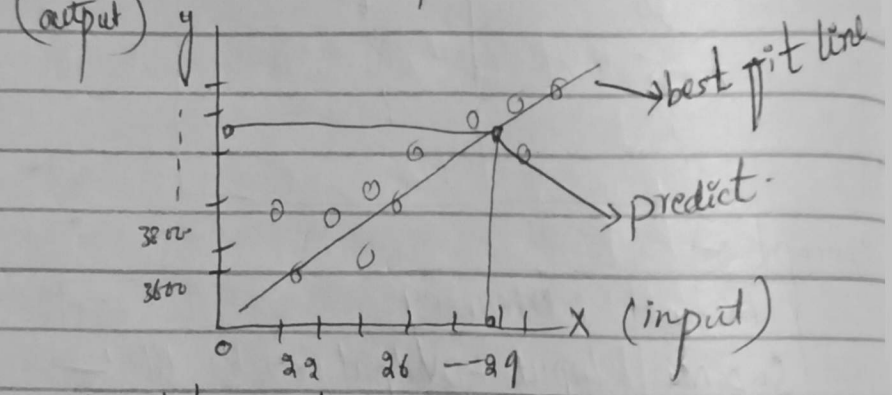
Linear Regression

Problem Statement

Input → output

Age	Salary
22	36000
26	38000
28	40000
34	42000
34	82000
⋮	⋮

plot



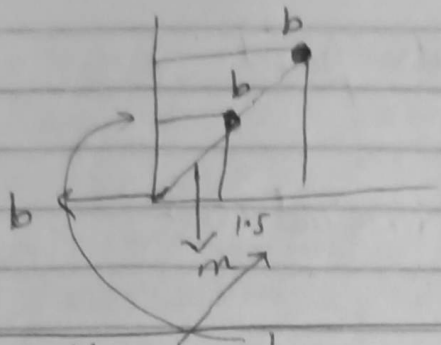
⇒ Now let see how you actually creat a best fit line.

$$y = mx + b$$

m → slop
b → intercept
x → data point
y → prediction

m :- x ↑ ले, तो y ↑ ले, तो तो step slop ↑

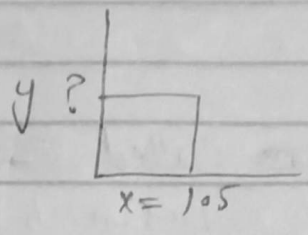
b :- पूरा line ↓ पूरी start ↓ ले, तो



Q1 $y = mx + b$ ——— $m = 1$, $b = 0$

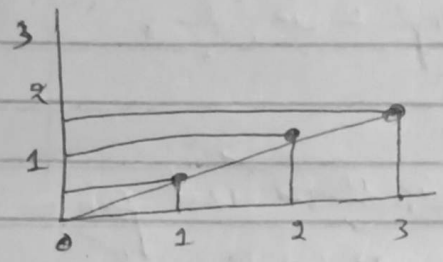
$y = 1(1.5) + 0$

$y = 1.5$



Q2 $m = 0.5$

$b = 0$



Q2 $y = mx + b$

$y = 0.5(1.5) + 0$

$= 0.75$

~~(Prediction → Probability)~~

$h\theta(x) = \theta_0 + \theta x$

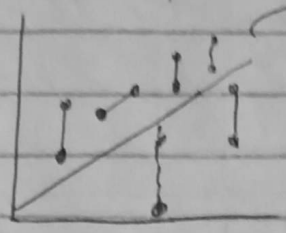
↓

$y = mx + b$

$h\theta \Rightarrow$ Intercept

$\theta_1 \rightarrow$ slope

Difference b/w actual Data point & predicted Data point.



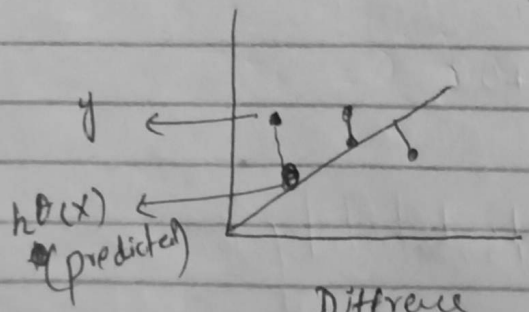
⟶ => Called Residual error.

sum of error ہی ہے ←

Residual error کو کم کرنا ہے تاکہ fine line مل سکے۔

Cost function :-

Residual error کو کم کرنے کے لیے cost fun ← sum کرتے ہیں



Difference

$$J(\theta_0, \theta_1) = h\theta(x)^{(i)} - y^{(i)}$$

Difference square

$$(h\theta(x)^{(i)} - y^{(i)})^2$$

$$\sum^{m} \frac{1}{2m} (h\theta(x)^{(i)} - y^{(i)})^2$$

sum of ہی ہے ← 2m

you have to minimize the cost func by changing the value of θ_0, θ_1

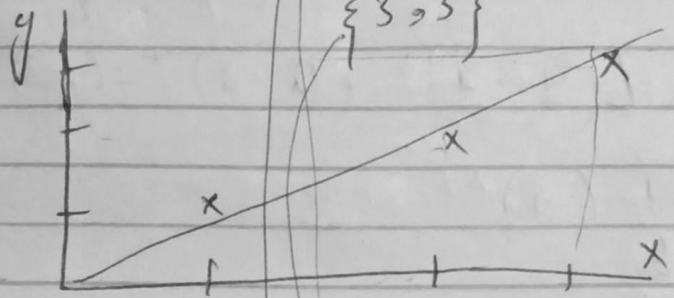
Gradient Descent:-

Data point x y

{1, 1}

{2, 2}

{3, 3}



x → Scatter plot

/ → best fit line.

$\theta_0 \Rightarrow 0$

$\theta_1 = 1$

$$h\theta = \theta_0 + \theta_1 x$$

$0 + 1(1) = 1$

$0 + 1(2) = 2$

$0 + 1(3) = 3$

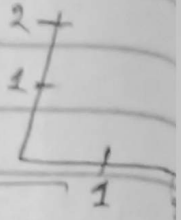
Residual error :-

$$J(\theta_0 - \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} (h\theta^{(i)} - y^{(i)})^2$$

put Value

$$\frac{1}{2m} \times \text{sum point} \left(\frac{\text{sum}}{0}\right) (3)^{\text{p.w}}$$

$$= 6m$$

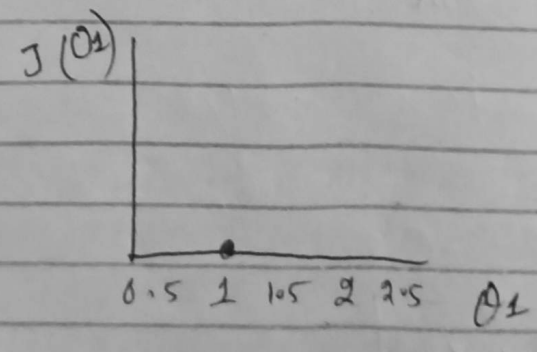


$$= \frac{1}{6} [(1-1)^2 + (2-2)^2 + (3-3)^2$$

$$= \frac{1}{6} (0)^2$$

$$= 0$$

$J\theta_0 = 0$ Ans



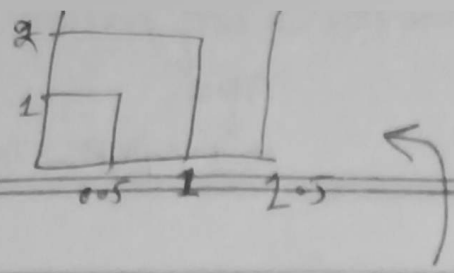
(Diffent line)

$$(h\theta) = \theta_0 + \theta_1 x_1$$

$$0 + 0.5\,(1)$$

$$0 + 0.5\,(2)$$

$$0 + 0.5\,(3)$$



$$J(\theta_1) = \frac{1}{6}\left[(1-0.5)^2 + (1-2)^2 + (3-1.5)^2\right]$$

$$= 0.58.$$

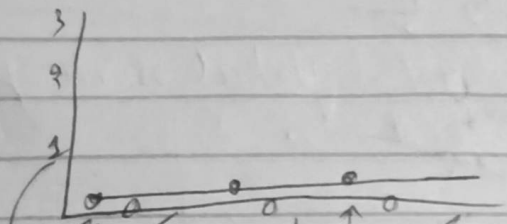
$\theta_1 = 0$, $\theta_0 = 0$

$h(\theta) = \theta_0 + \theta_1 x_1$)

$0 + 0 = 0$

$0 + 0 = 0$

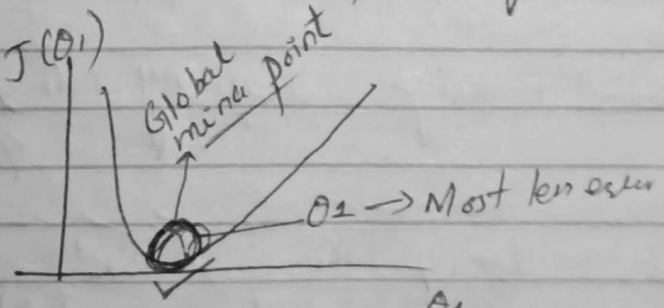
$0 + 0 = 0$



distance ← Residual error

$$J(\theta_1) = \frac{1}{6}\left[(0-1)^2 + (0-2)^2 + (0-3)^2\right.$$

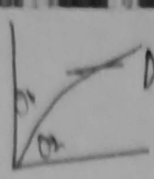
Note- Multiple θ_1 ka Value ko change kr ke



θ_1 ki value ka cost fun ko min kia

(steps, we follow)

① Main Data point



find PE? → $(PE)^2$ → cost function

GDC

find GMP → less error?

(2) θ_1 & θ_2 کی value use کر کے ایک line create کرتے ہے

(3) اس line پر Residual error کو find کرتے ہے

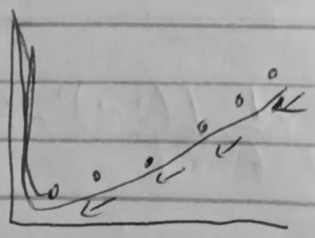
(4) ان سب کو Mean Squear error میں ڈالتے ہے

(5) یہ mean square error / cost function بناتا ہے

(6) اس cost func کو Grad desc curve میں convert کرتے ہے

(7) اس curve میں سے global mina point کو find کرنا پڑتا ہے

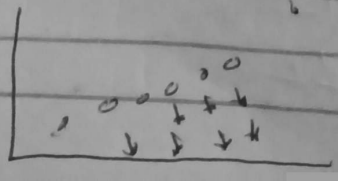
$-b'' \leftarrow$ Least error کی طرف

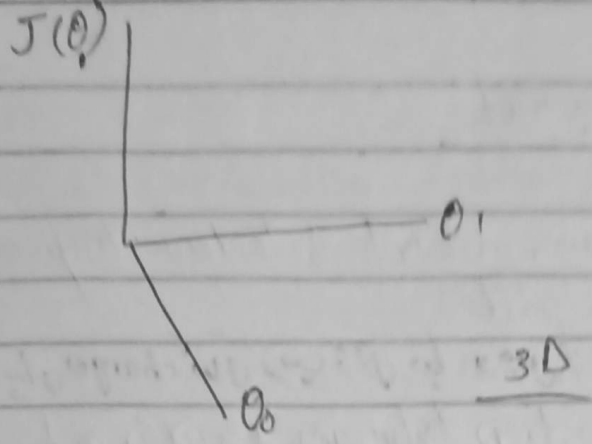


Grad Des میں

Learning rate ← ~~Height~~ کو low رکھتے ہے

جواب:- کیونکہ اگر height یا learning rate زیادہ ہوگا تو loss function ← $J(\theta)$ کا value زیادہ اوپر نیچے آسکتی ہے اسی لیے learning rate کو کم رکھتے ہے تاکہ آہستہ آہستہ نیچے جاسکے





3D

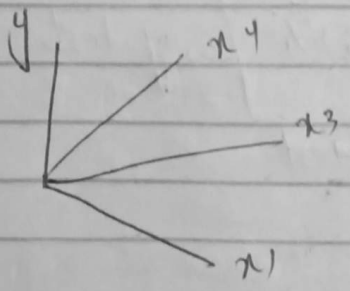
Multiple linear Regression

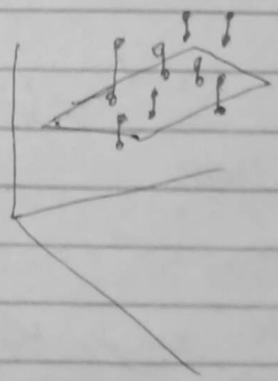
$y = mx + b$

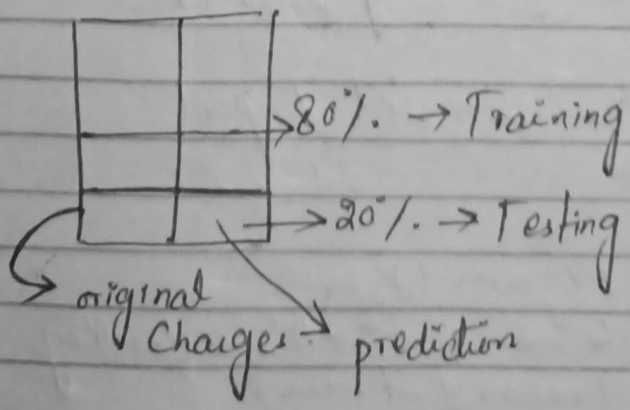
$h(\theta) = \theta_0 + \theta_1 x_1$

In Multiple LR :-

$h(\theta) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 \dots \theta_n x_n$







Train-Test Split

library :

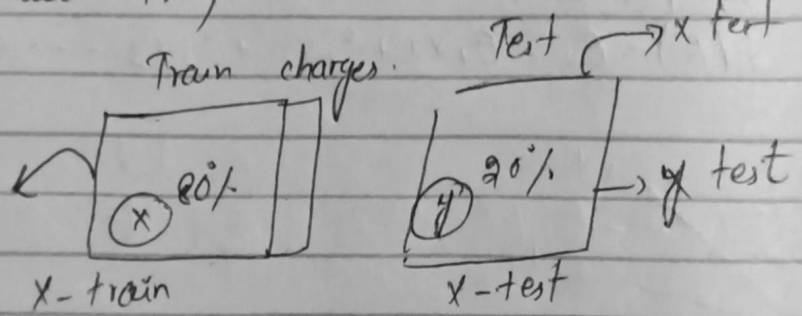
```
from sklearn.model_selection import
train_test_split
```

ستون charges کل دیتا x میشه

```
x = final_df.drop('charges', axis = 1)
y = final_df['charges']
```

Variable. ستون charge دیتا y میشه

```
x_train, x_test, y_train, y_test =
train_test_split(x, y, test_size=0.33, random_
state = 42)
```



↓

prediction x کل. prediction میشه این
compare بعدا y و این

```
from sklearn.linear_model import
LinearRegression
```

```
model = LinearRegression()
```

create یک model از lin reg

```
model.fit(x_train, y_train)
```

fit x train y را به model

Test کرنے کا کتنا اچھا کام کر رہا ہے

Model Evaluation technique :-

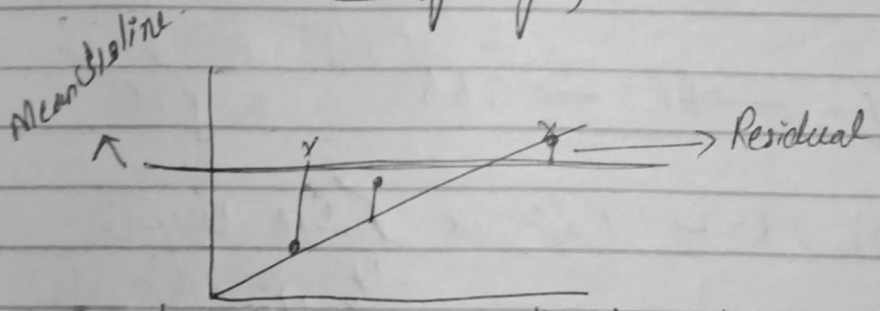
performance Matrices کہتے ہیں :-

- R^2
- Adjusted R^2

- یہ بتاتا ہے کہ model کتنا اچھا perform کر رہا ہے ← R^2

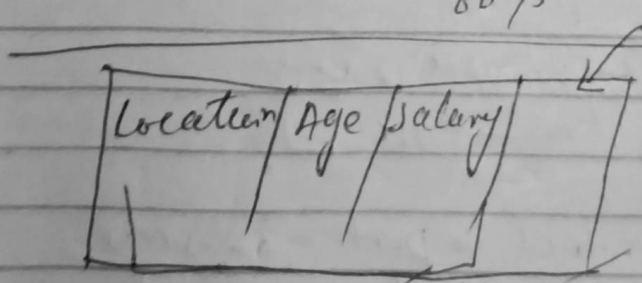
$$R^2 = 1 - \frac{\text{Sum of Residuals}}{\text{Sum of total}}$$

$$1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - y_i)^2} \rightarrow h(\theta)$$



Note :- The result will be always + ve
→ The result will be in decimal.

Result => 0.86
86%



$R^2 = 81\%$
$R^2 = 88\%$

by add ی sleep ~~feature~~ جب

$R^2 = 89\%$

Adjusted R^2 :-

$$1 - \frac{(1-R^2)(n-1)}{N-P-1}$$

n = no of Rows
R = Realtek

Age - Salary

Age → R^2 → 81%.
Salay → AR^2 → ~~85~~ 79%.
Age + location → R^2 → 89%.

sleep + → AR^2 → 86%.

AR^2 کم

بہتے دیکھا sleep کو use کرنا چاہیے R^2 تو زیادہ ہوا مگر AR^2 زیادہ بہتر ہوا۔

Practice → prediction

- y_pred = model.predict(X_test)
- y_pred.

ان سب کو compare کرنے کے لیے use

Model Evaluations :-

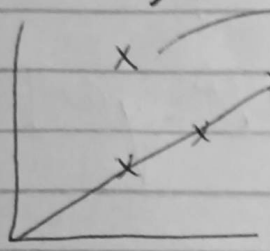
- from sklearn.matrics import r2_score
- r2 = r2_score(y_test, y_pred)

Result : 80.80 40

↳ accuracy 80% یعنی model کا

formula:
- n = X_test.shape[0]
- p = X_test.shape[1]
- adjusted_r2 = 1-((1-r2) * (n-1)/n-p-1)
- adjusted_r2.

Overfitting & Underfitting



(low biase) (high bias)

Note: training → Model perform [illegible] (Good)
testing → model Not perform Good.
↳ (high Variance) (high Varia)

average

M1	M2	M3
Train → 92%	Train: 82%	Train - 68%
Test → 64%	Testing: 90%	Test - 64%

[illegible] Test [illegible] best [illegible] Train [illegible] Result
[illegible] Overfitting [illegible]

[illegible] underfitting [illegible]

Training → 92% → perfect Model.
Testing → 90%.
↘ low bias & low Variance

Training → 68%
Testing → 64% → underfitting.
↘ high bias & Variance

(Ridge & Laso Reg)

(Ridge)

Cost function =
↳ add کرنے کے لئے (0) output میں

$$J(\theta_0 - \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} [(h\theta_x^{(i)} - y^{(i)}] + \lambda (slop)^2$$

$= \emptyset$

(Laso)

↳ mode کا slop میں format کیا

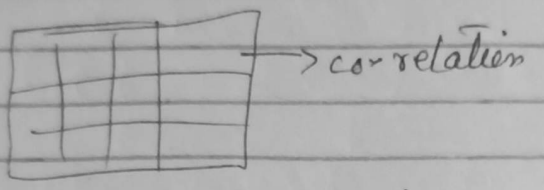
(Practice)

1. df.shape / describe / head / isnull.sum
2. EDA :-
   sns.histplot(df[]) → bins
   ↘ kde
3. check co-relation
   ↳ Related کے لئے different color ہوتا ہے

   df.corr (numeric_only = True)

ML lecture 2
shayan school

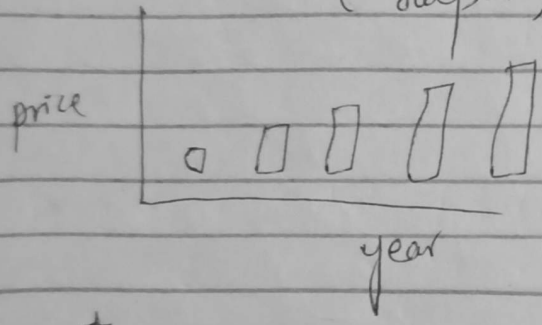
sns.heatmap(df.corr(numeic_only=True), annote=True)



(4) year ↔ price کے بیچ میں co-relation ہے

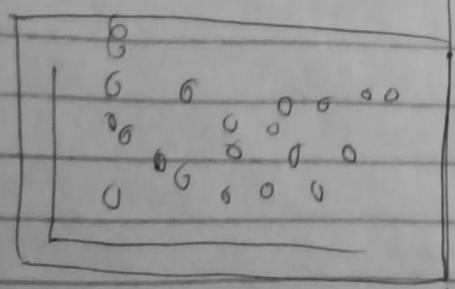
sns.boxplot(data=df, x='year', y='price')

(output)



~~pairplot(relation)~~

(5) sns.scatterplot(data=df, x='milage', y='price')



(6) graph میں x اور y میں Result کے df.color کا رنگ ہے

/ ایک column کو الگ کرنا

```
X = df.drop(colum = ['price'], axis=1)
y = df['price']
X
```

we one hot Encoding

```
X_encoded = pd.get_dumies(X.colums[]=[,])
```

one hot

جو column کو 0 اور 1 میں convert کرتا ہے

Result

before 34 colm ⟶ after 38 colm.

Label Encoding ← convert ← Categor Data

```
from sklearn.preprocessing import LabelEncoder
encode = LabelEncoder()
coulm = [' ', ' ', ' ']
xlabel = X
```

— { loop }

```
for i in columns
```

~~xlabel~~ ← fit کرنا

```
xlabel[i] = encode.fit(xlabel[i])
```

▷ `df.[].value_counts[]`

Scaling ← stdrd Scalr use کرنا ہے

```
from sklearn.preprocesg import standscaler
numeric cols = [' ' ' ']
scaler = Standrdscaler()
X-one-encd(nomric-cols = scales.fit
                    -trasform
```

4. Encode Column

▷ X label. colon

output [(), (), ()]

▷ ylabel [, ,] = scaler. fit-tra

[(), (), ()]

- encoding ht 1, 0 - जैसे encode करने पर Result

(One hot encoding)

Import → from sklearn model-selection import train-

→ linear model import Linear

→ metrics import mean-absolute-error, mean Square error, r2-

▷ X-train, X-test, y-test, y-test_split

(X-one-encode, y test_size=0.

random state =

▷ model = LinearReg()

model. fit (X-train, y-train)

▷ y-pred = model. predict (X-test)

y-pred. → compare ← output

▷ y-test

use r^2

▷ r^2 = r2_score (y-test, y-pred)

r2.

output = 0.83 ←

▷ adjusted r2?

```
n= X_test.shape[0]
p= X_test.shape[1]
adjusted_r2 =1-((1-r2)*(n-1)/(n-p-1)
print("adjusted R2 score:", adjusted r2)
```

output :- 0.83 --

(Label Encoding)

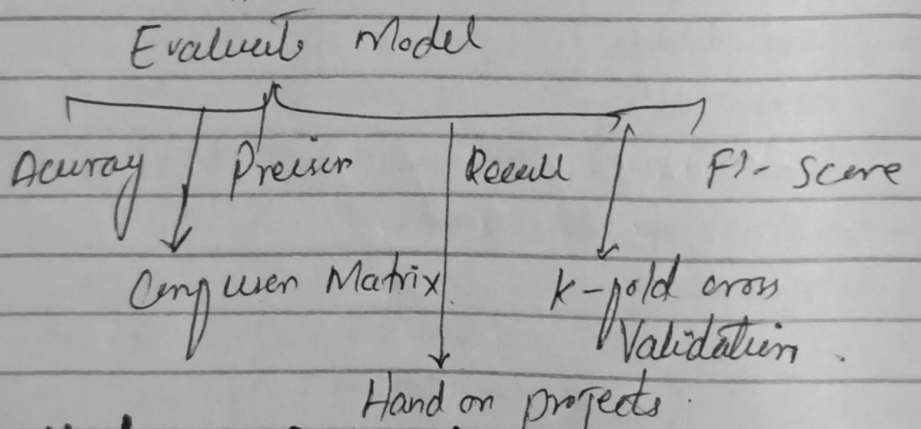
label: ~~ج~~ One hot میں rule ~~ہے~~ same

Lect : 03 => Classifications

Goal

Classify data into Categories with multiple Algorithms.

1. What is Classifications
2. Logistic Reg
3. K-nearest Neighbour (KNN)
4. Decision Tree
5. Random forest Classifier
6. Naive Byes.
7. SVM. (Support Vector Machine



What is Classifications?

A Classification algorithm is a type of Machine learning alg used to predict a category or class Label for a given input.

&

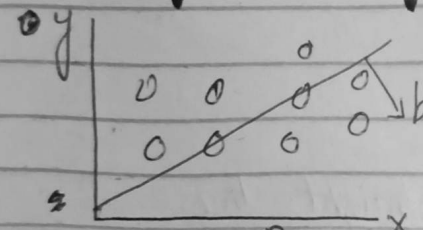
Classification is the process of teaching a Machine to answer question like : "Is this is a cat or a dog. or "Will customer survive or not"

output

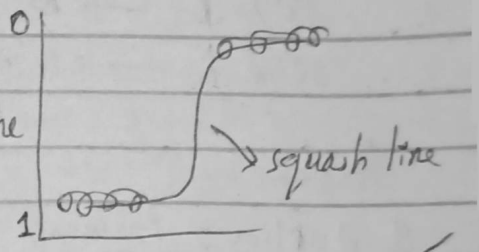
Ex 2

weyt	obes
20	No
29	No
30	No
60	Yes
80	yes

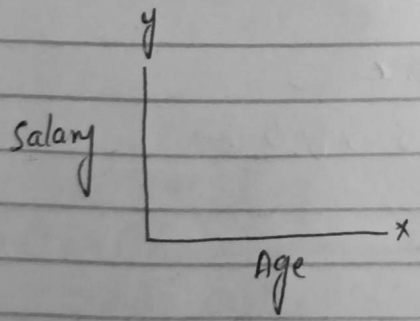
Logistic Regression

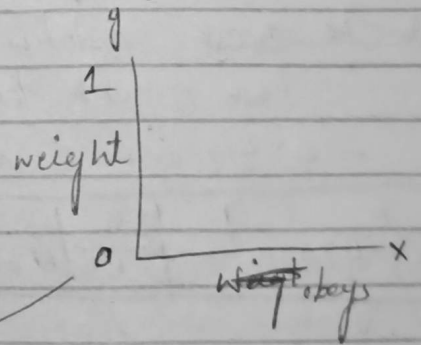


Linear Reg

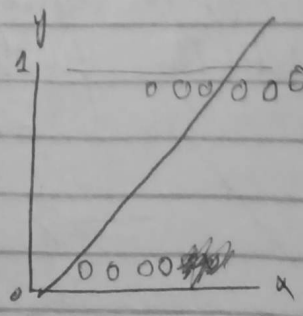


logistic Reg (obeys)



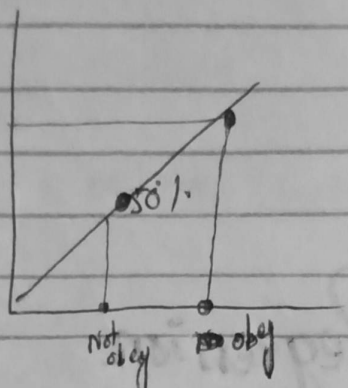


weyt	obeys
92	Y
63	N
24	N
102	Y
196	Y



To create the best fit line :-

$h\theta(x) = \theta_0 + \theta_1 x_1$



50% ↓ → Not obeys.

50% ↑ → obeys.

Problem :-

① Outlier : Influence the Straight line.

② Result can be 0 & 1.

↓

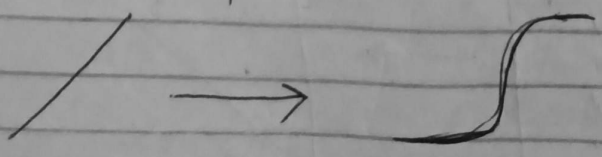
~~So to prove~~ Solution

we Squash the line

- ← → curve ∫ shap S ∫ الـ ←

Sigmoid function

To Make the Straight line into ∫ shape curve we will use SIGMOID activation function.



formula -

$$h\theta(x) = g(\theta_0 + \theta(x_1))$$

$$g = \frac{1}{1+e^{-z}}$$

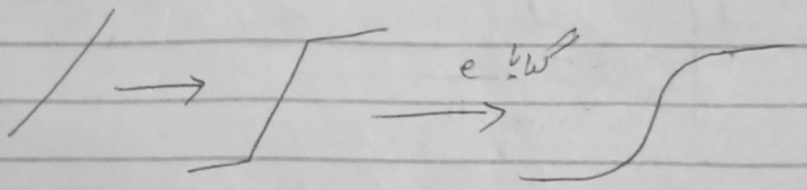
(put g in formula)

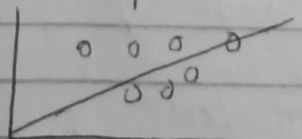
$$h\theta(x) = \frac{1}{1+e^{-(\theta_0+\theta_1 x_1)}}$$

$$h\theta_{(x)} = \frac{1}{1+e^{-(\theta_0+\theta_1 x_1)}}$$

e → eulers number ; Mathmteal constant
Value : 2.71828
Use : curve smoth کرنا

e → given smoth curve.



Linear Reg	Logistic Reg
→ θ_0, θ_1 → Values → to create best fit line	θ_0, θ_1 → Value use to create best squash line
→ use : cost function	→ Use : Log Loss function
	Loss کو سب کو sum کرنا
→ Data point سے Main point تک difference کو find کرنا ہے	

CamScanner

→ find Residual

→ ई Residual के sum

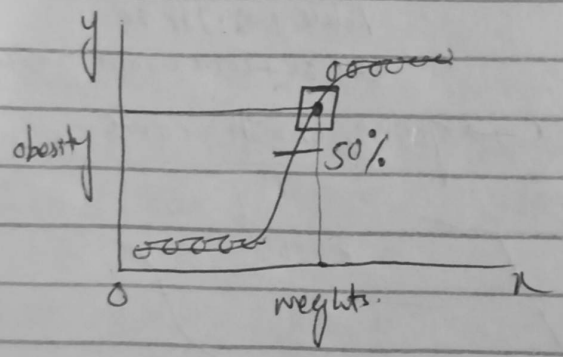
→ Result called error.

→ ई error के graph पर

ऐसे ई ओर Grad

Desc पर put करें ई

Log Loss (Binary Cross Entropy)



(0, 1)

★ Let say Predided prob → 0.99 and actual is 1 ↓

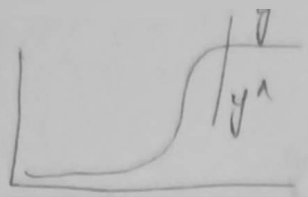
we are Very less Loss.

★ Let say pred prob = 0.01 & actual is 1. ↓

Very Heigh Loss.

Called panelty

Note: Penalty is Calculate with



formula :-

$$\log \text{Loss} = \frac{-1}{m} \sum_{i=1}^{m} [y^{(i)} \log(\hat{y}^{(i)}) + (1-y^{(i)}) \log(1-\hat{y}^{(i)})]$$

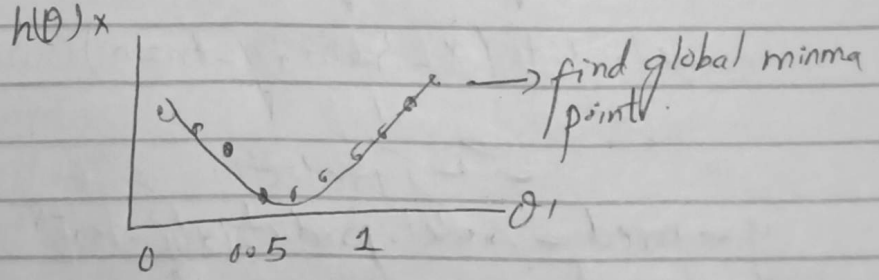
m = number of sample
y = actual Label (0-1)
y^ ⇒ predicted probability (0-1)

Find ⇒

① yi = 1
y^i = 0.54

log loss = 0.616.

② ← ← (Sumber) Σ
③ Graph Creat ←.



⑦ Code Implementation :-
⑧ Model Evaluation

Import Data ←
null value ←
drop color ی Null ← object
df.fillna() ← plane B ←

Mean Med Mode
Fill ←

string

object / float → Label encoder ←

like : Male, femal

0 1.

code :

le.fit_transform()

df = df.astype(int)

X = df.drop[ ] ← Label y

y = df[ ]

Train - Test Split ←

Model ←

from sklearn.linear_model Import Log

mode = LogisticRegression()

model.fit(X_train, y_train)

prediction ←

y_pred = model.predict(X_test)

y_pred.

Real data ←

y_test

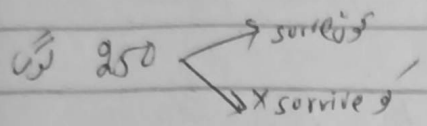
model

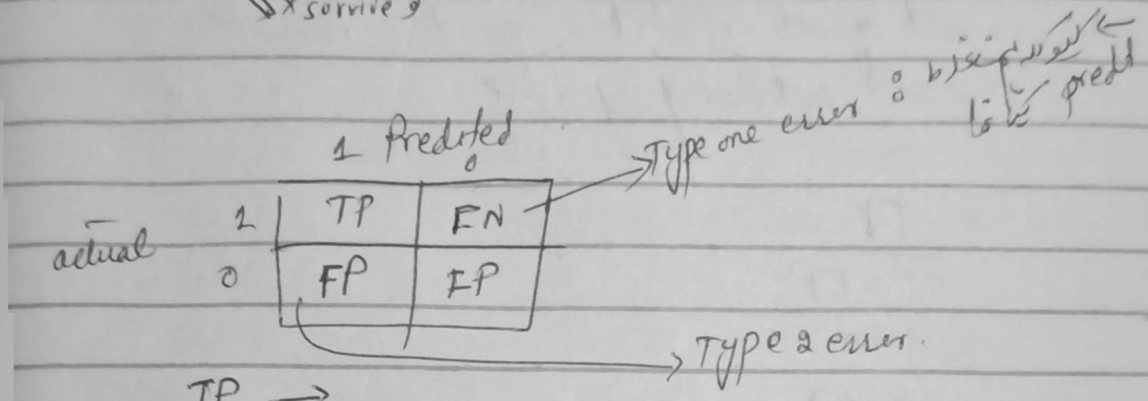
use Confusion Matrix

Evaluation

Confusion Matrix :-

predict 0 & 1.





TP →

FP → ?. predict کیا تھا غلط تھا

190	10
10	40

(Predict)

(1) Accuracy
(2) Precision
(3) Recall
(4) F1 Score

(i) Accuracy

→ The percentage of correct prediction

$$= \frac{TP + FP}{TP + FP + FN + FP}$$

$$= \frac{190 + 40}{190 + 10 + 10 + 40}$$

$$= 0.92 \qquad 92\%$$

(2) (Precision)

- out of all predicted positive how many were actually positive.

$$= \frac{TP}{TP+FP}$$

$$= \frac{190}{190+10}$$

$$= 0.95 \quad 95\%$$

- Note: Use when false positive costly (spam detection).

(3) Recall

Out of all actual positive. how many did we correctly predicted.

true positive / false negative

$$= \frac{TP}{TP+FN}$$

$$= \frac{190}{190+10}$$

$$= 0.95\%$$

use when false Negative are costly (disease detection).

ex: کینسر کی تشخیص میں corona کی تشخیص

(ⓦ F1 Score)

→ Harmonic mean of precision and Recall.

$$F1\ score = \frac{2 \times precision \times Recall}{precision + Recall}$$

Uses when you want balance b/w precision & Recall.

ex:- دونوں میں stability کے لیے

Practice:-

▷ from sklearn.matrics import accuracy_score, confusion_matrix, Classification_report

accuracy F1score --

▷ accuracy_score (y_test, y_pred)

▷ confusion_matrix (y_test, y_pred)

output:

array [90 14] → wrong
[10 53]

▷ classification report (y_test, y_pred)

output: | 4 ro

accuracy F1 score

print ... output ... row ...

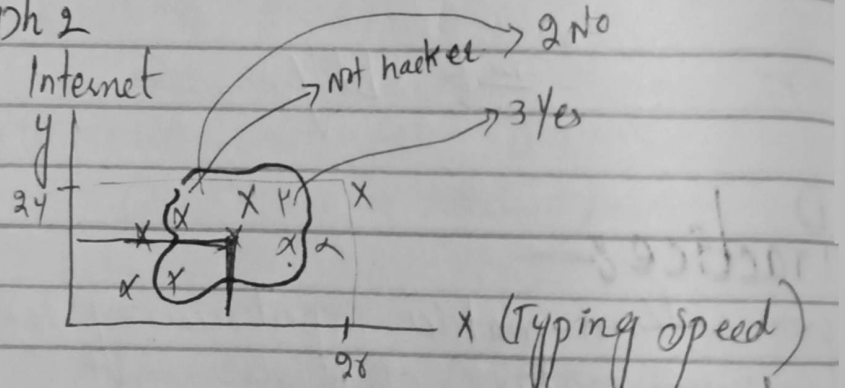
(kNN) — Predict logistic ... Continuous Variab...

K-Nearest Neighbour.

... use ... KNN ... feature scaling ...

Internet	Typing speed	Hacker
24	56	1
12	22	0
⋮	⋮	⋮

Graph 2



① Define k value → k=5 → hyperparameter
(5 nearest neighbors)

② Calculate the Distance from all points (euclidean Distance).

③ Sort all the Distance → 3 yes, 2 No

④ Count Majority → yes person become hacker

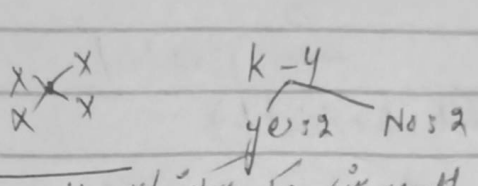
(Other than Classification)

→ Majority ... Mean ...

- 5 ← nearest ← neighbor ← mean ← K
- you can use N-Dimensions.

(Rules)

① K → K value must be odd number



Use کریں odd اس لیے اس سے result میں کوئی مسئلہ نہ ہو

② Experiment

→ Cross Validation.

2000 input

1600 Train — 400 Testing

K=5	86%
K=3	89%
K=1	
K=7	68%

K=25 90%

K=25 میں 90% سے زیادہ آیا تو ہم نے رکھ لیا — Note

(Practice)

→ standard scaler (1)

scale · fit · transform (2)

↙ - ↙ import sklearn ③

▷ from sklearn.neighbors import KNeighborsClassifier

▷ model = KNC()

▷ fit

↙ predict ④

KNN-model.Predict(-)

↙ accuracy score ⑧

accuracy-score(--)

Confusion Matrix ⑥

⑦

print(classification-report(y-test, y-pred-knn))

(Naive Bayes Theorem)

Independent Variable	Dependent Variable

ex 1 —

Dice = P(6) = 1/6

P(4) = 1/6

✓ 1/3 ↙ effect

ex 2 —

coin P(H) = 1/2

P(T) = 1/2

Deck of card :- Total 52

52

26(R) 26(B)

P(R) = 26/52 = 1/2.

P(R) = 25/51 ?

effect ✓

$P(A \text{ and } B) = P(A) \times P(B)$

$= \frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$

$P(A \text{ and } B) = P(A) \times P(B/A)$

conditional probability

Derive Bayes Theorm :—

$P(A \text{ and } B) = P(B \text{ and } A)$

① $P(A) \times P(B/A) = P(B) \times P(A/B)$ ②

$$P(A/B) = \frac{P(A) \times P(B/A)}{P(B)}$$ called Bayes Theorm

* x_1, x_2, x_3, x_4 → Independet events

y → depend (output feature)

Bayes Theorm ؟ و! و= Douve !

$$P(A/B) = \frac{P(A) \times P(B/A)}{P(B)}$$ (put bayes in formula)

$$P(y/x_1 x_2 x_3 x_4) = \frac{P(y) \times P(x_1 x_2 x_3 x_4 / y)}{P(x_1 x_2 x_3 x_4)}$$

← ↳ class y و= task ؟ و! ←

$$P(\text{yes}/x_1 x_2 x_3 x_4) = \frac{P(y) \times P(x_1/y) \times P(x_2/y) \times P(x_3/y) \times P(x_4/y)}{P(x_1) \times P(x_2) \times P(x_3) \cdots P(x_n)}$$

ا. feats ←

confuse Matrix

$P(x_n|y)$

NB → $P(N|x_1, x_2, x_3, x_4) = \frac{P(N) \times P(x_1|N) \times P(x_2|N) \cdots}{P(x_1) \times P(x_2) \cdots P(x_n)}$

? Test Data

(Practice)

وہ Naive bys میں Std Scales کو فرورت نہیں ہوتی
جو فرورت ہوتی ہے وہ KNN میں ہوتی ہے

```
from sklearn.naive_bayes import GaussianNB
model_NB = GaussianNB()
```
→ model کا نام

```
#model کو fit
model.NB.fit_transform.
model_NB.fit(x_train, y_train).
```
normal distribution ہے

```
y_pred_NB = model_NB.predict(x_test)
```

یہ ہمارا Prediction کتنا اچھا ہے

accuracy score, recall, f1-score, support

یہ سب کو call کرنے کے لیے

```
print(classification_report(y_test, y_pred_N
```

(Decision Tree)

Tree → Branches
leave
Root

ex 2

If my money is not known

If money < 30 : → snak

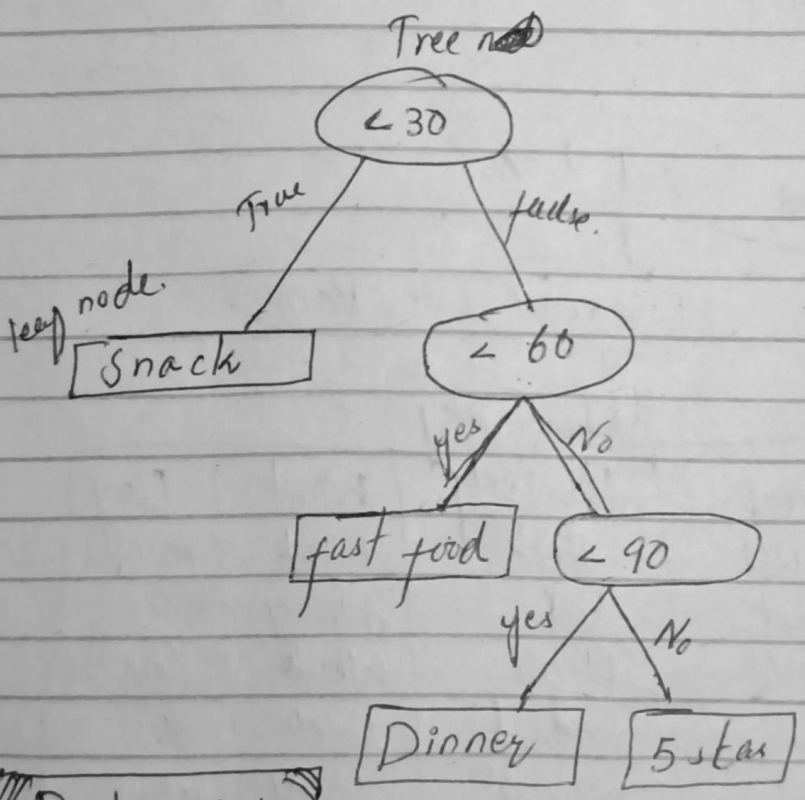
money > 30 & < 60 : → fast food

elif money > 6 & < 90 : → Dinner

else :

5 star

Tree Structure



Data Set

(Practice)

Search of w/b Docum

```
from sklearn.naive_bayes import GaussianNB
X-train
model-NB = GaussianNB()        #Model
model-NB.fit_transform
model-NB.fit(X-train, y-train)
```

y_pred_NB = model_NB.pred(x_test)

y_pred_NB

confusion Matrix

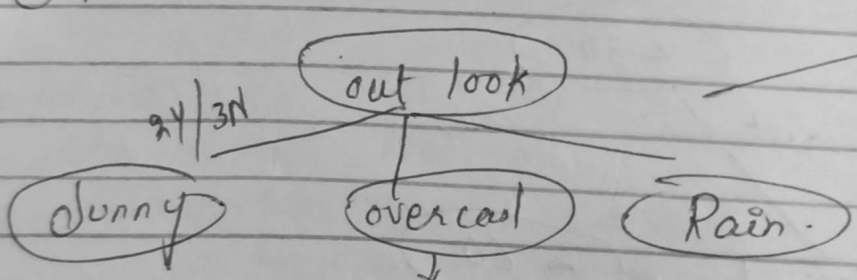
▷ Confusion Matrix (y_test, y_pred_NB)

Prec / Recall / f1 score / Support

▷ Print (classification report (y_test, y_

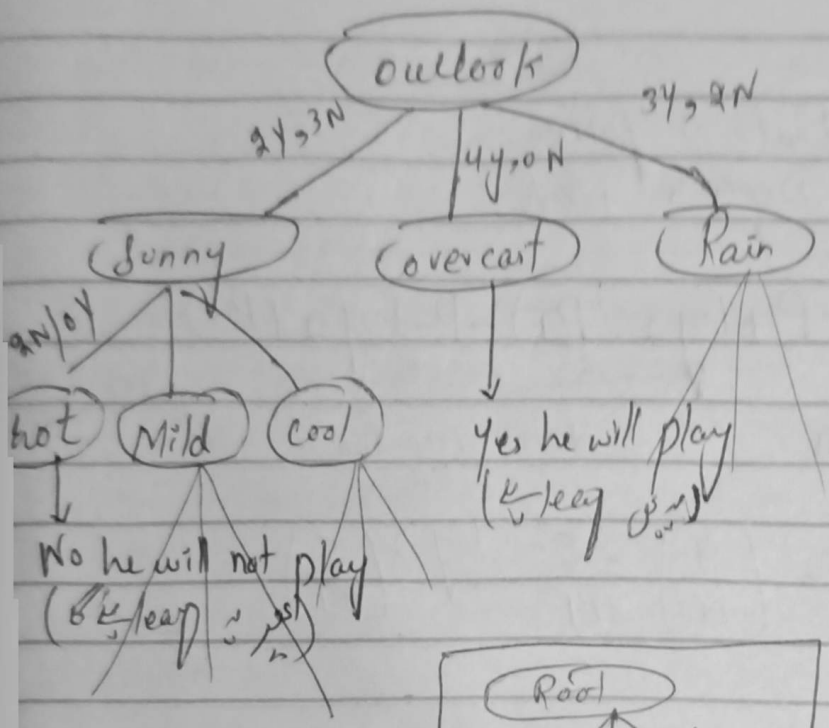
(Use Data Set to Create Decision Tree)

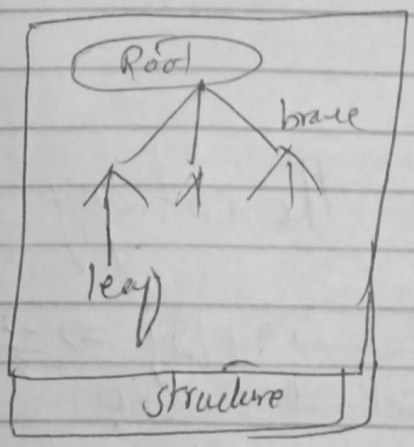
(1) Sunny
(2) Overcast
(3) Rain



outlook	Temp	Humidity	Wind	Play
Sunny	Hot	High	Weak	No
Sunny	Hot	H	Strong	No
Overcast	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No

Sunny	3 No, 2 Yes
Overcast	4 Yes, 0 No
Rain	3 Yes, 2 No





Entrop, Information
for choosing Root node.
gain ← information ← consider ← root node

How we choose Root node?

Structure ←
1- start with Data
2- Choose the best feature
3- Make branches
4- Repeate the process for each branch

⑤ Stop when :
All Data is pure
More Depth is reach

entropy (s) = $-P_+ \log_2(P_+) - P_- \log_2(P_-)$

Play = 5 nodes positive proportion

$\frac{-9}{14} \log_2\left(\frac{9}{N}\right) \frac{-5}{14} \log_2\left(\frac{5}{14}\right)$

⇒ 0.940

→ (Calculate the entropy of outlook)

→ Sunny → 2y/5y ⇒ $\frac{-2}{5} \log_2\left(\frac{2}{5}\right) - \frac{3}{5}\log$
→ Overcast ⇒ 4 0 & 4 = 0 & entropy = 0.971
→ Rain ⇒ 0.971 ← after calculation.

→ (Information Gain)

True feature

★ IG (S, outlook) = entropy (S) − Σ |S|

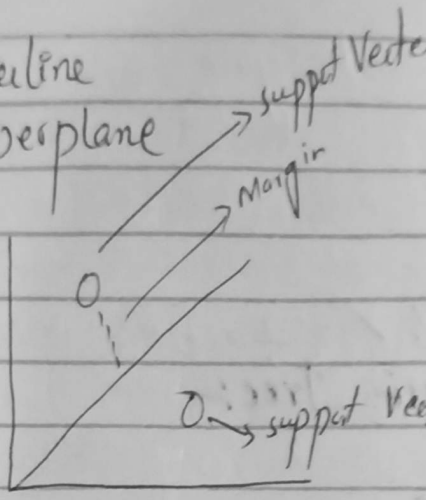
Total feature ← |Sv|

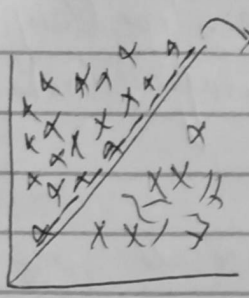
Entropy |Sv|

0.940 − [$\frac{5}{14}$ × 0.971 + $\frac{4}{14}$ × 0 + $\frac{5}{14}$ × 0.

↳ son bhi 3

2D → hyperline
3D → hyperplane





- Supp Vec ۔ b ۔ Touch ۔ margin lin

$$x w + b = 0$$

x => point you want to predict
w => weight / slope.
b => Bias

- ۔ use ۔ predict ۔ b ۔ line

$wx + b > 0$ → Class 1 → Red
$wx + b < 0$ → Class 2 → Green

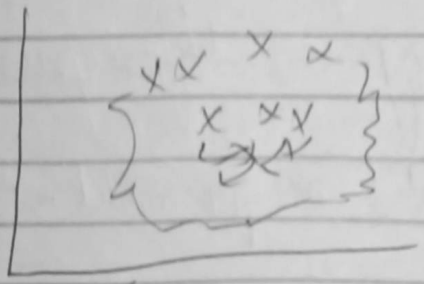
Question
What is hyperplane?
A line that seprate 2 categories

kernals : To Distinguish b/w Red & Green points

Q: What is margins?
The Space b/w the line & nearest point

Q: What is Support Vector
The closest ball to margin that support the line

Q: Dimension?



hyperpla ... Solution

change ... Dimension ... kernals

Implementation of SVM

```
from sklearn.svm import SVC
model_svm = SVC()
                kernal='rbf'
# fit - train + feature Scaled
# [Classification]
# prediction → X test-scaled
# Accuracy Score
# Confusion matrix [[ ]]
# Classification report
```

Lec : 04

1- Model Tuning
2- Ensemble learning
3- Unsup Learn
- K - Means
- Elbow Method
- Hierachal Clustring
- PCA (Dimentonality Reducter Basics)
4. Hands on project

~~(Model Tuning)~~ / Hyperparam

~~Model 1~~

performance best کے model ML کی ہم adjust کرتے ہیں setting کو model کے ML

setting کی دو قسمیں ہوتی ہیں model کے ML آپ کو Ex :like دیتی ہیں

max_depth (کتنا deep کتنا Tree)

main_samples_split

(کتنا boosting کرنا ہے) learning_rate

—: Problem

overfit کرتا ہے model کتنا depth زیادہ

underfit کرتا ہے model کتنا depth کم

—: reset

best result کتنا کتنا try کرنا ہے value آپ

select کرنا ہے final میں

(Types of tuning)

① Manual Tuning:
value کی try کرنا ہے

② Grid Search:
check کرنا ہے combination

③ Random Search

Ex

test ← ہے جس میں کم ہو
test ← ہے جس میں زیادہ

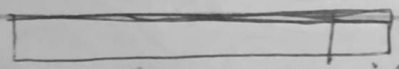
Tuning ⇐ ہے adjust کرنا ہوتا ہے test ← perfect

Importance

- Accuracy Improve
- Overfitting ↓
- Model reliable ہوتا ہے

(Cross Validation)

Train test split



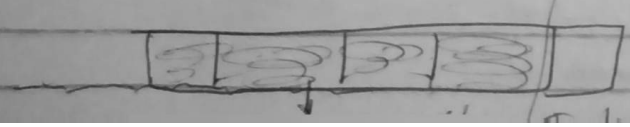
80% ⟶ Training Data
20% ⟶ Testing
⇒ 20% ⟶ Train 80% پر fit کیے جائے

- Overfit
- Underfit
- poor performance

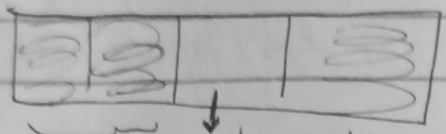
کیا ہو سکتا ہے

K-fold cross Validation

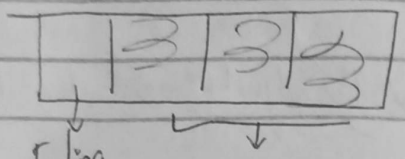
ہے جس میں data کو K



→ 5 parts کو ہم



Accuracy 82



Accuracy 86%

Results —

You Divide the Data in k part &
every part will be your testing Data
Remaining will be your training Data

↳ every 6 % کے ہم

Average = 83 %.

Practice

linear

KNN

▷ from sklearn.model-selection import cross-val-score.

▷ Scaler = StandardScaler()
X-Scaled = Scaler.fit(Training) Model

▷ score = cross-val-score (model-svm, X-scaled, y, cv = 5, scoring=

▷ print (scores).
(0.83---)

Average
Print (scores.mean())

output :-
>0.82--

(Hyper parameter)

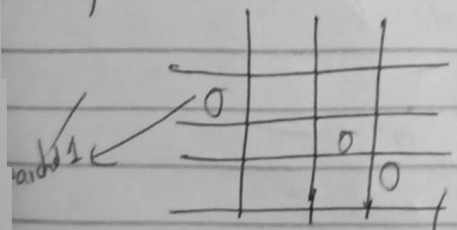
- Ridge & Laso Regression = α = Learning rate
- Decision Tree → max depth, Sample split, max features.
- KNN → [3, 5, 7, 9]
- SVM → kernal
- Random forest → DB scane.

~~Manual Search~~

Methods

1) Manual Search
2) Grid Search
3) Randomized Search

△ Grid Search CV → cross Validation



~~from sklearn~~

△ from sklearn.model_selection import GridSearchCV
△ Classifier = GridSearchCV(model - svm), {
'C': [1, 10, 20, 30]
'kernel': ['rbf', 'linear'
CV = 5, return_train_score=False)

Classifier.fit(X, y)

#print

▷ classifier.cv_results.

▷ result = pd.DataFrame(classifier.cv_resu

▷ results.['', '',

9 KNN Neighbours

KNN SVM Naïve Bayes

Steps

① Data preprocess
② Model overfit / underfit
③ Cross Validation
④ Grid.
⑤ They you will create perfect mode

(Random Search CV)

ex :

KNN

N = 6

weight = 2

Metric = 2

(X G Best)

10 X 2 X 10 = 2000

Learning rate = 10

n estimator = 20

maxdepth = 10

(Practice)

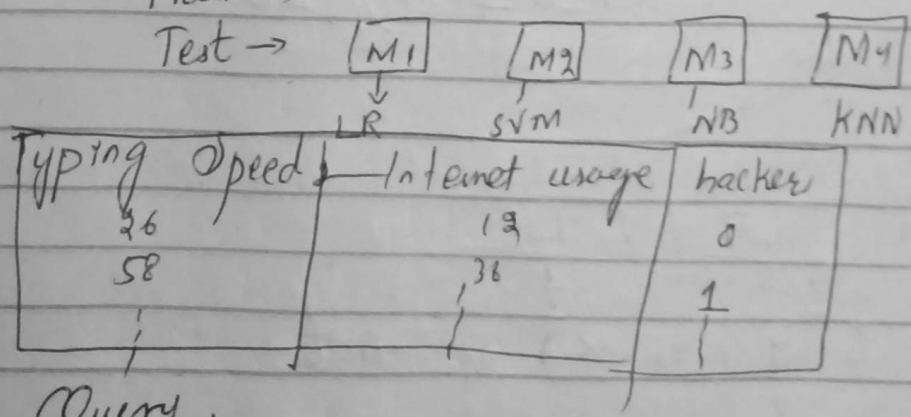
```
from sklearn.model_selection import
                    RandomsearchCV
Classifier = (GridSearch RandomSearchCV), {
        'C' : [1, 10, 20, 30],
        'kernal' : ['rbf', 'linear'],
    }, cv = 5, return_train_score = False
    n_iter = 10

Classifier_r.fit(X, y)
results = pd.dataframe(classifier_r.cv_results)
results[' ', ' ', ' ', ' ']
```

(Ensamble Learning)

Q :- We hear the crowd best [illegible] stars [illegible] open [illegible] amazon [illegible]

[crowd ∝ best]



Query :-

Typing speed = 56

Iu = 28

- NO Wi 14ei Wi model ← 3
- 6 ½ ½ ix hacker ooo

Regression :-
Mean instead of Voting.

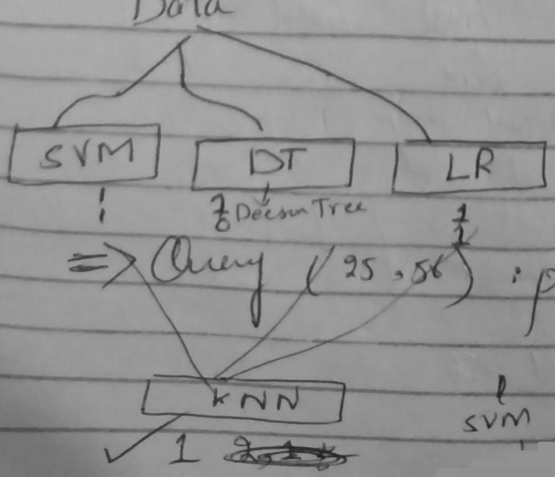
Types of Ensemble Learning

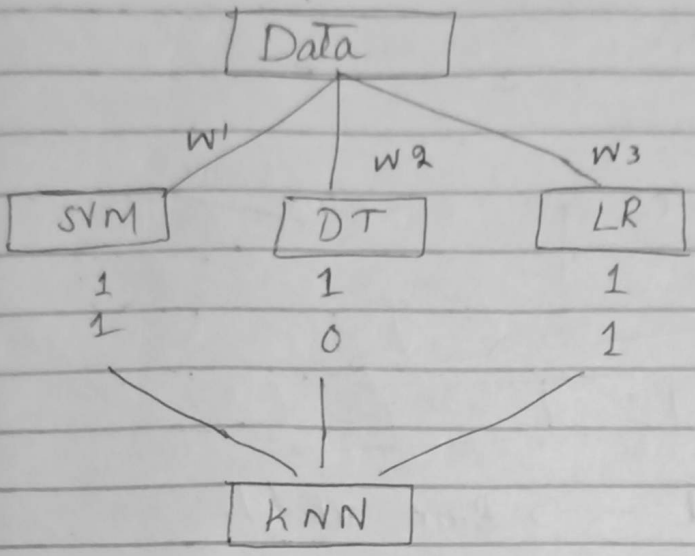
① Bagging → Random forest.
② Boosting → Ada boost
③ Stacking → Gradient bossting
→ XG Boost.

(1)
Bagging

(3)
Stacking

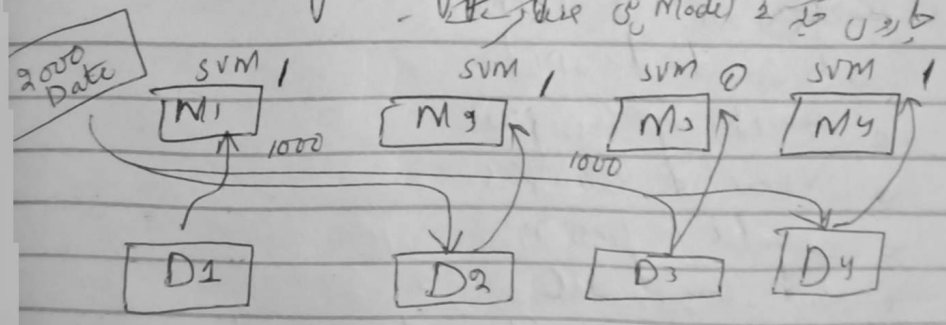
ex :- کیس کے دو لڑکیں جیسے Chairs





Bagging

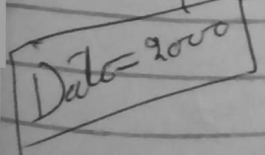
↳ Boosting Aggregation



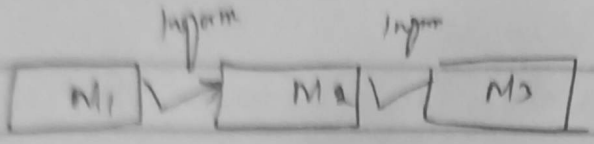
final output = 1

Random forest ←

(Boosting)



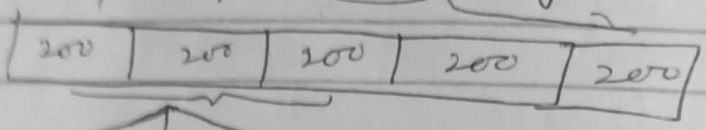
Ada Boosting & XG Boosting is Best.



Practice

1000 points → Testing → Metalearner

Training



SVM, LR, DT → Base Model.

Metamodel → 200+200 = 400

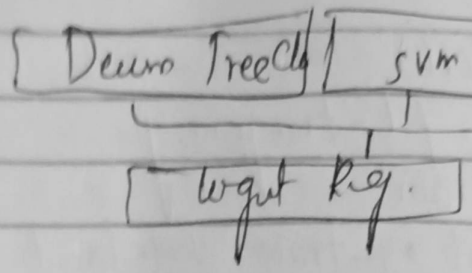
(Practice)

Import sklearn.
- train_test split
- Stacking Classifier
- DecisionTree Classifier
- Logistic Regression
- svm Import SVC
- accuracy_score, Classification_report
- Import seaborn as sns.

```
df = sns.load_dataset('iris')
# Data cleaning
# کالم کو X میں اور کالم کو y میں specie
X = df.drop('specie', axis=1)
y = df['species']
# Label encoding
```

```
le = LabelEncoder()
y_encoded = le.fit_transform(y)
    # test-train
X_train, X_test, y_train, y_test = train_test
    _split (X, y_encoded, test_size = 0.
    2, random_state = 42, stratify = y_encoded)
        use, model ko jo jo #
Base_learners = [
    ('dt', DecisionTreeClassifier(random_state = 42)),
    ('svc', SVC(probability = True, kernal = 'rbf',
                random_state = 42))
    ('lr', LogisticRegression(max_iter = 1000))
]
# meta learner ⟶ k / Logistic Reg
meta_learner = logistic Regression (max_iter = 1000)
    estimators = base_learners,
    final_estimators = meta_learner,
    cv = 5

stacking_clf = stackingClassifier(
    estimators = base_learners,
    final_estimator = meta_learner,
    cv = 5
)  \ cross Valid.
        fit #
stacking_clf.fit(X_train, y_train)
    output
```



▷ y_pred = stacking_clf.predict(X_test)
▷ accuracy = accuracy_score(y_test, y_pred)
▷ accuracy
0.981 ——— Good

▷ from sklearn.neighbors import KNeighborsClassifier
▷ model_knn = KNeighborsClassifier(n_neighbors
▷ model_knn.fit(X_train, y_train)
see score
▷ model_knn.score(X_test, y_test)
0.98

Note: When you change your parameter then they diagnostically affect your model.

▷ from sklearn.svm import SVC
▷ model_svm = SVC(C=30, kernal='rbf', gamma='auto')

• C is punishment

penalty کا ایک Decision Tree کے side پر hyperparameter

▷ model_svm.score(X_train, y_train)
▷ 0.98

(Change)

X

```
from sklearn import SVC
model_svm = svc (C=10, kernal='linear',
    gamma='auto')
model_svm.fit (x_train, y_train)
model_svm.score (x_test, y_test)
output → 0.1
```

use Grid Search CV

```
from sklearn.model_selection import
    GridSearchCV
Classifier = GridSearchCV(
    'C': [1, 10, 20, 30]
    'kernal': ['rbf', 'linear']
    }
    cv = 5)
# print
classifier.cv_Results
```

Data
↓
Model
↓
CV
↓
Grid

} Perfect Model

(Random Search CV)

KNN

N = 6
weight = 2
metric = 2

= 6 x 2 x 2
= 24

▷ when it was 24. It was handle able

XG Boost

Learning rate = 10
n estimator = 20
Max Depth = 10

10 x 20 x 10
2000 → dpp parameter.

▷ when it was 2000 not handlable.

out of 2000 - choose 24 randomly

(Practice)

▷ from sklearn.model_selection import RandomizedSearchCV

▷ Classifier = GridSearchCV(model_svm), {
'C' : [1. ...]
'kernal' : [—]
CV = 5, return train score = false)

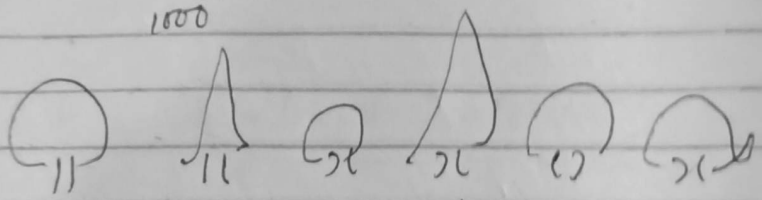
iter = 4

Bagging

ایک ہی model کو use کرنا۔

LR , LR , LR

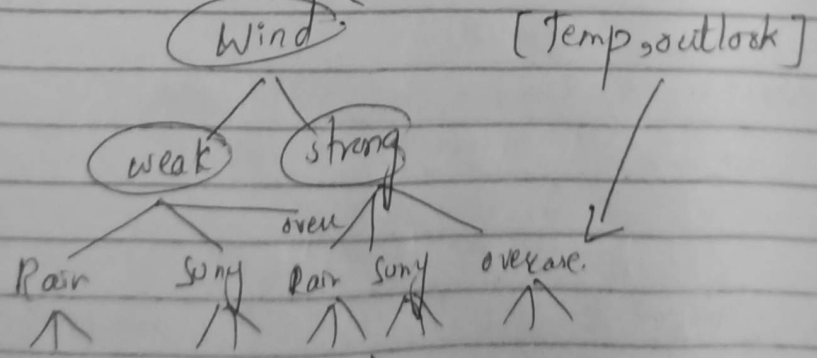
1000 → 800



ہر ایک Tree کو different sample دیا جاتا ہے۔

⇒ use Random forest Classifier.

- Make 100 decision tree
- 1000 → 600 کا sample
- Root node → [Temp, Wind]

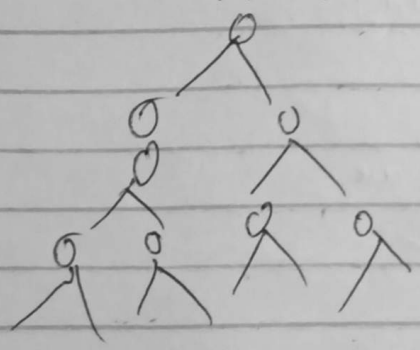


Practice

```
rf-model = RandomForestClassifier(
n_estimators = 100
max_depth = None,
random_state = 42
)
```

```
rf_model.fit(X_train, y_train)
#Predict
y_pred = rf_model.predict(X_test)
#Accuracy
accuracy = accuracy_score(y_test, y_pred)
accuracy
```

Bagging



Train Best
Testing Poor

- Low Bias
- High Variance → Called Overfitting

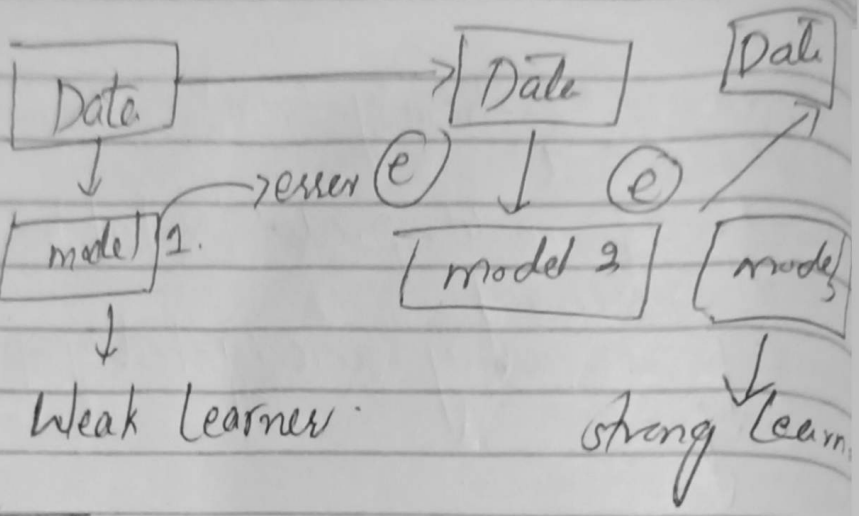
↓ Invert

Low bias
Low Vance (Generalize)

Boosting → High biase, high Vance

↓ Inverted

Low biase, Low Vance



$$f = \alpha_1(m_1) + \alpha_2(m_2) + \alpha_3(m_3) \quad \alpha_n(m_n)$$

Baggine

① Ada boost → Less use

② Gradient Boosting → Less use

③ XG Boost ✓ Most use

Practice

```
from sklearn.ensemble import
AdaBoostClassifier, GradientBoosting
Classifier.
from xgboost import XGClassifier
ada-model = AdaBoost Clas
(n-estimators=100, random-stat=42)
# fit model
# predicter
# accuracy score
```

0.9833

gb_model = GradientBoostingClassifier
(n_estimators = 100, Learning_rate = 0.1,
random_state = 42)

~~y_pred = gb_model.predict(.fit~~

fit
predict.
xgb_model ----

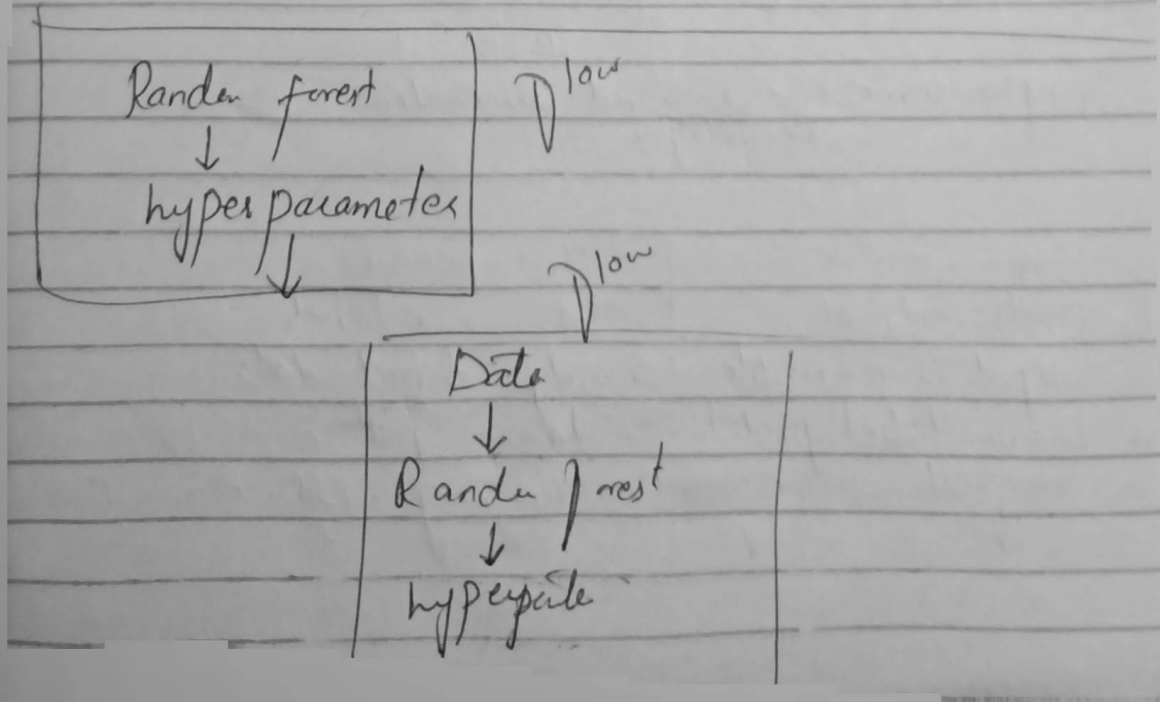
test-train.
accuracy.

(Boosting Advantage)

62% Model accuracy score

↓

80% Model Accuracy score.



Unsupervised ML

→ Not Label Data
→ find patterns, structure & relationship in the data by itself.

=> Goals :-

ایک ڈیٹا سے ہم کیسے ڈیٹا سے seard کو pic
ڈیٹا سے ایک سے cluster کرے
ہے اور کون سے clustering alg یہاں پر

- ہے Dimentionality Reduct کے use کو PCA
کرنے کو کون

ایک ٹرانزیکشن میں bank → outlier/anomaly
Rs
ایک 30k کے 2500 میں
میں 1lack کے اس پر چوری کا
آ سکتا ہے

Netflix same series کو find associate
Recreation

Supervise	Unsupervise
• Data → label	• Data → x label
• output is available	• only Input data
• Learn label predict	• learn pattern in D
• classification & regres	• Clustring, DR, AR

Clustring

Grouping similar Data points.

Why use Unsup Learning?

- To explore & understand data structure
- To discover hidden pattern.
- To reduce complexity of data.
- To find anomilies or outliers
- To Processes & Improve Sup model.
  e.g → PCA + classifier.

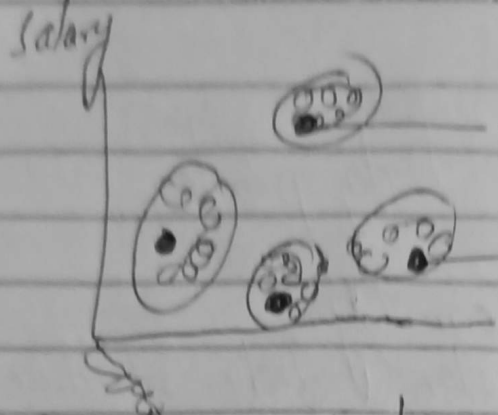
uses :-

99% work ⟶ Sup
1% ⟶ unsup

- Customer Segmentation
  گوگل ایسے ویسے ads & phone کرتا ہے
- Market Segmentation
  زیادہ بیچا جائے ⟵ بندہ
- Anomaly Detection
  fraud - & bank
- Image Compression
  → reduce Image size.

(Clustering) :-

K-mean Clustring :-



(1) Decide n clusters
(2) Initialize centroids → center point
(3) Assign cluster
↳ Eucledian Distance (cabite distance).
(4) Reassign the center point
center ↳ new cluster
↳ Calculate distance
(5) finish.

Note

method ↳ elbow ↳ Default 100 ↳ use

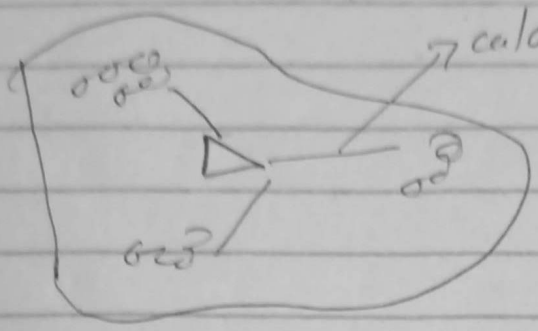
Elbow Method :

(y) WCSS

(x) number of cluster

wcss → within cluster sum of square distance.

K=1



$d_1^2 + d_2^2 + d_3^2 \text{ --- } d_n^2 - wcss$

~~4 clusters~~ (3 ; 00 : 00)